🌌 NEBULA LANGUAGE

> *Um interpretador completo para uma linguagem de programação didática com sintaxe em português e características únicas.*

🎯 Sobre o Projeto

A **Linguagem Nebula** foi desenvolvida como trabalho prático para a disciplina **N477 - Construção de Interpretadores e Compiladores** do Centro Universitário do Triângulo (Unitri).

Este projeto implementa um interpretador completo que demonstra os conceitos fundamentais de:

- **Análise Léxica**: Tokenização do código-fonte
- **Análise Sintática**: Geração de AST com precedência de operadores
- **Análise Semântica**: Verificação de tipos e escopo
- **Interpretação**: Execução da árvore sintática abstrata

✨ Características

🔤 Sintaxe Original

A linguagem Nebula implementa **6 variações** em relação ao exemplo de sala:

Aspecto	Exemplo de Sala	Nebula	Status
Palavras-chave	`if`, `else`, `while`, `let`, `print`	`se`, `senao`, `enquanto`, `criar`, `exibir`	✅ Variado
Delimitadores de bloco	`:` e indentação	`inicio` ... `fim`	✅ Variado
Delimitador de statement	Nova linha	`;`	✅ Variado
Operadores lógicos	`and`, `or`, `not`	`&&`, ``	`, !`
Declaração de variável	`let` / `var`	`criar`	✅ Variado
Comentários	`#`	`//` e `/* */`	✅ Variado

🛠️ Funcionalidades Implementadas

- ✅ Declaração e atribuição de variáveis
- ✅ Expressões aritméticas (`+`, `-`, `*`, `/`, `%`, `^`)
- ✅ Operadores relacionais (`==`, `!=`, `<`, `<=`, `>`, `>=`)
- ✅ Operadores lógicos (`&&`, `||`, `!`)
- ✅ Condicional de 2 ramos (`se` / `senao`)
- ✅ Repetição (`enquanto`)
- ✅ Saída de valores (`exibir`)
- ✅ Comentários de linha e bloco
- ✅ Recuperação de erros sintáticos
- ✅ Precedência e associatividade de operadores

📦 Instalação

Pré-requisitos

- Python 3.7 ou superior
- Nenhuma biblioteca externa necessária

🚀 Uso

Comando Básico

```
python src/nebula.py examples/tests<arquivo.neb>
```

Opções

```
# Modo verbose (mostra tokens e AST)
python src/nebula.py examples/programa_exemplo.neb --verbose

# Ajuda
python src/nebula.py --help
```

Exemplo de Execução

```
# Executar programa exemplo
python src/nebula.py examples/programa_exemplo.neb
```

📝 Sintaxe da Linguagem

Olá Mundo

```
exibir("Olá, Mundo!");
```

Declaração de Variáveis

```
// Declaração simples
criar x;

// Declaração com inicialização
criar y = 10;
criar nome = "Nebula";
criar ativo = verdadeiro;
```

Condicionais

```
se (condicao) inicio
    // comandos
fim
senao inicio
    // comandos
fim
```

Repetição

```
enquanto (condicao) inicio
    // comandos
fim
```

Operadores

Tipo	Operadores
Aritméticos	`+`, `-`, `*`, `/`, `%`, `^`
Relacionais	`==`, `!=`, `<`, `<=`, `>`, `>=`
Lógicos	`&&`, `
Atribuição	`=`

Comentários

```
// Comentário de linha única

/*
 * Comentário de bloco
 * Pode ter múltiplas linhas
 */
```

💡 Exemplos

1. Cálculo de Média

```
criar nota1 = 7.5;
criar nota2 = 8.0;
criar media = (nota1 + nota2) / 2;

se (media >= 7.0) inicio
    exibir("Aprovado");
fim
senao inicio
    exibir("Reprovado");
fim
```

2. Fatorial

```
criar n = 5;
criar resultado = 1;
criar i = 1;

enquanto (i <= n) inicio
    resultado = resultado * i;
    i = i + 1;
        fim

exibir(resultado);
```

3. Operadores Lógicos

```
criar a = verdadeiro;
criar b = falso;

se (a && !b) inicio
    exibir("Condicao verdadeira");
fim
```

🏗️ Arquitetura

```
┌──────────────────┐
│  Código Fonte    │  .neb
│   (Texto)        │
└──────────────────┘
         │
         ▼
┌──────────────────┐
│ Analisador       │  Tokenização
│    Léxico        │  (Lexer)
└──────────────────┘
         │
         ▼
┌──────────────────┐
│     Tokens       │  Lista de tokens
│                  │  com tipo, valor,
│                  │  linha e coluna
└──────────────────┘
         │
         ▼
┌──────────────────┐
│ Analisador       │  Geração de AST
│   Sintático      │  (Parser)
│   (Parser)       │  com precedência
└──────────────────┘
         │
         ▼
┌──────────────────┐
│       AST        │  Árvore Sintática
│                  │  Abstrata
└──────────────────┘
         │
         ▼
┌──────────────────┐
│  Interpretador   │  Execução
│                  │  (Evaluator)
└──────────────────┘
         │
         ▼
┌──────────────────┐
│     Saída        │  Resultado
```

```
│                    │
└────────────────────┘
```

Componentes Principais

Componente	Arquivo	Descrição
Lexer	`nebula.py` (classe `AnalisadorLexico`)	Tokenização do código-fonte
Parser	`nebula.py` (classe `Parser`)	Análise sintática e geração de AST
AST	`nebula.py` (classes `ASTNode`)	Nós da árvore sintática
Interpreter	`nebula.py` (classe `Interpretador`)	Execução da AST

🧪 Testes

O projeto inclui 5 arquivos de teste:

Arquivo	Descrição	Propósito
`teste_precedencia.neb`	Testa precedência de operadores	Cobertura técnica
`teste_condicional.neb`	Testa estruturas se/senao	Cobertura técnica
`teste_erros_recuperaveis.neb`	Testa recuperação de erros	Erros recuperáveis
`teste_invalido.neb`	Código com erros inválidos	Teste de erros
`programa_exemplo.neb`	Programa não-trivial	Demonstração

Executar Todos os Testes

```
# Teste de precedência
python src/nebula.py examples/tests/teste_precedencia.neb

# Teste condicional
python src/nebula.py examples/tests/teste_condicional.neb

# Teste de recuperação de erros
python src/nebula.py examples/tests/teste_erros_recuperaveis.neb

# Teste inválido (espera-se erros)
python src/nebula.py examples/tests/teste_invalido.neb

# Programa exemplo completo
python src/nebula.py examples/programa_exemplo.neb

# Executa todos
python python src/run_tests.py
```

📚 Documentação

A documentação completa está disponível em:

- 📄 **ESPECIFICACAO_NEBULA.pdf** - Especificação técnica completa
  - Gramática BNF/EBNF
  - Tabela de tokens
  - Precedência e associatividade
  - Sintaxe de blocos e comandos

🎓 Conceitos Acadêmicos Demonstrados

Este projeto demonstra os seguintes conceitos da disciplina:

Aula 1 - Gramáticas

- ✅ Definição de gramática em BNF/EBNF
- ✅ Símbolos terminais e não-terminais
- ✅ Regras de produção

Aula 2 - Análise Léxica

- ✅ Implementação de lexer
- ✅ Enum de tipos de token
- ✅ Reconhecimento de lexemas
- ✅ Tratamento de erros léxicos

Aula 3 - Análise Sintática

- ✅ Parser recursivo descendente
- ✅ Precedência de operadores
- ✅ Associatividade (esquerda e direita)
- ✅ Recuperação de erros (modo pânico)

Aula 4 - Análise Semântica

- ✅ Tabela de símbolos
- ✅ Verificação de tipos
- ✅ Escopo de variáveis
- ✅ Geração de código intermediário (conceito)

Aula 5 - Execução

- ✅ Interpretação da AST
- ✅ Ciclo de execução
- ✅ Avaliação de expressões

👨‍🏫 Professor

Prof. Me. Jair de Oliveira Pereira Neto
Disciplina: N477 - Construção de Interpretadores e Compiladores
Centro Universitário do Triângulo (Unitri)

✨ Agradecimentos

- Centro Universitário do Triângulo (Unitri)
- Prof. Me. Jair de Oliveira Pereira Neto
- Colegas da turma de Ciência da Computação

🌌 **NEBULA LANGUAGE - 2026**
Construção de Interpretadores e Compiladores